

ОТЧЁТ

по лабораторной работе № 3

**ПАРТИЦИОНИРОВАНИЕ POSTGRESQL: РАЗДЕЛЕНИЕ БОЛЬШИХ
ТАБЛИЦ И АВТОМАТИЗАЦИЯ СОЗДАНИЯ ПАРТИЦИЙ**

по модулю «Масштабирование баз данных»

Исполнитель:

обучающийся группы K0709-23/3

подпись, дата

У. И. Усачева

Руководитель:

преподаватель

подпись, дата

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Партиционирование учебных таблиц	8
1.1 RANGE-партиционирование по дате.....	8
1.2 Partition pruning	9
1.3 RANGE-партиционирование по числовому значению	10
1.4 LIST-партиционирование.....	11
1.5 Неизвестное значение и DEFAULT-партиция	12
1.6 HASH-партиционирование	13
1.7 Выбор стратегии партиционирования	14
1.8 Партиционирование и индексы	15
1.9 Когда партиционирование не помогает.....	16
2 Автоматизация создания и контроля партиций	18
2.1 Автоматическое создание партиций	18
2.2 Сбой ночной задачи: проверка и оповещение	20
3 Партиционирование базы данных сервиса TaskManager.....	24
3.1 Шаги 1–4. Таблица, ключ, стратегия и обоснование	24
3.2 Миграция таблицы.....	25
3.3 Шаг 5. Партиции	26
3.4 Шаг 6. Запросы API и partition pruning	26
3.5 Шаги 7–9. Автоматизация, контроль и alert.....	29
4 Ответы на контрольные вопросы	30
ЗАКЛЮЧЕНИЕ	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
ПРИЛОЖЕНИЕ А Миграция таблицы tasks на партиции.....	37

ПРИЛОЖЕНИЕ Б Задача создания партиций и проверка.....	39
ПРИЛОЖЕНИЕ В Правила отправки оповещений.....	43

ВВЕДЕНИЕ

Работа продолжает лабораторную работу № 2, в которой было показано, что при росте таблицы до миллионов строк одного индекса становится недостаточно. Следующий шаг — разделить большую таблицу на части по естественному признаку данных: по дате, категории или хэшу ключа.

Цель работы — научиться проектировать и эксплуатировать партиционированные таблицы PostgreSQL: выбирать стратегию и ключ партиционирования, проверять `partition pruning` по планам выполнения, автоматически создавать будущие партиции, контролировать их наличие и оповещать о сбое.

Для достижения цели решались следующие задачи:

- исследовать RANGE-, LIST- и HASH-партиционирование и DEFAULT-партицию на учебных таблицах, проанализировать планы выполнения запросов с ключом партиционирования и без него;
- оценить совместную работу партиционирования и индексов и цену большого числа партиций;
- реализовать в сервисе TaskManager задачу создания партиций, проверку их наличия и оповещение в Telegram без повторов и с сообщением о восстановлении;
- смоделировать сбой ночной задачи и восстановление системы;
- перевести таблицу задач сервиса на партиции и проверить реальные запросы API.

Работа состоит из трёх частей: в разделе 1 эксперименты выполняются над учебными таблицами в схеме `lab03`, в разделе 2 описана автоматизация создания и контроля партиций, в разделе 3 — партиционирование таблицы `tasks` сервиса. При выполнении использовались документация PostgreSQL [1–4], документация ASP.NET Core [5] и Telegram Bot API [6], методические материалы [7, 8] и инструмент записи терминала VHS [9].

Таблица 1 – Среда выполнения экспериментов

Параметр	Значение
СУБД	PostgreSQL 16.14 (postgres:16-alpine) в Docker Desktop, Windows 11
Ресурсы Docker	20 CPU, 10,7 ГБ памяти
shared_buffers/work_mem	128 МБ / 4 МБ
Сервис	TaskManager: .NET 8, EF Core 8, Dapper, Liquibase, Redis
Данные сервиса	5 000 001 задача за 25 месяцев (2024-09 ... 2026-09), 3 984 786 связей задача-тег

Все эксперименты автоматизированы:

Таблица 2 – Скрипты и результаты

Часть	Запуск	Результаты
1–9 — учебные таблицы в схеме lab03	bash scripts/lab-03/run-sql-part.sh (скрипты 00–03)	docs/lab-03/results/0*.txt, measurements.csv
12 — запросы сервиса к tasks	bash scripts/lab-03/run-service-part.sh before (до миграции 012) и ... after (после)	docs/lab-03/results/10-service-*.txt, 11-api-queries.txt
12 — SQL, который строит EF Core	bash scripts/lab-03/capture-api-sql.sh	docs/lab-03/results/api-sql.log
10–11 — job, проверка, alert	сервис TaskManager; сценарий сбоя: powershell -File scripts/lab-03/demo-alert.ps1	docs/lab-03/results/alert-demo.txt
Запись терминала	vhs scripts/lab-03/demo.tape	docs/lab-03/demo.gif

Как измерялось. Каждый замер делает функция `lab03.measure` (`scripts/lab-03/00-setup.sql`):

- 1) выполняет запрос вхолостую, чтобы данные попали в кэш;
- 2) получает план `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` и извлекает из него: сколько партиций осталось в плане, сколько из них реально выполнялось, `Subplans Removed`, тип сканирования, индексы, `actual rows`, `Rows Removed by Filter`, прочитанные буферы, `Planning Time` и `Execution Time`;
- 3) печатает текстовый план в лог.

Все замеры лежат в таблице `lab03.measurements`, из неё выгружается `measurements.csv`. Время — один прогон на «тёплом» кэше: абсолютные

значения зависят от машины, сравнивать нужно порядок величин и число прочитанных буферов (одна страница — 8 кБ).

Запись терминала. GIF снят инструментом VHS на работающем сервисе: пропадает будущая партиция, проверка отправляет alert в Telegram, повторная проверка молчит, job восстанавливает партицию, приходит recovery (части 10–11).

```
> powershell -ExecutionPolicy Bypass -File scripts/lab-03/demo-alert.ps1 -Phase break -Pause 2

=== Партиции lab03.events: сегодня + 3 дня вперед + 1 день запаса
events_2026_09_09, events_2026_09_10, events_2026_09_11, events_2026_09_12, events_2026_09_13, events_2026_09_14, events_2026_09_15

=== GET /health/partitions
HTTP 200 Healthy: tasks = 0k; lab03.events = 0k

=== Сбой: ночная задача не отработала, партиции events_2026_09_14 нет
DROP TABLE

=== GET /health/partitions
HTTP 503 Unhealthy: tasks = 0k; lab03.events = Critical: missing events_2026_09_14

=== PartitionHealthCheck: POST /api/partitions/check
tasks      0k      missing: -
lab03.events Critical missing: events_2026_09_14  alert: critical alert sent

=== Через 5 минут проблема та же: POST /api/partitions/check
tasks      0k      missing: -
lab03.events Critical missing: events_2026_09_14  alert: already notified
> █
```

Рисунок 1 – Сбой: партиции events_2026_09_14 нет, проверка возвращает CRITICAL и отправляет alert, повторная проверка alert не повторяет

```

> powershell -ExecutionPolicy Bypass -File scripts/lab-03/demo-alert.ps1 -Phase restore -Pause 2

=== Восстановление: CreatePartitionsJob, POST /api/partitions/ensure
tasks      0k      missing: -
lab03.events 0k      missing: events_2026_09_14   created: events_2026_09_14

=== Лог задачи в API
[21:26:50 INF] Partition job started at 2026-09-11 21:26:50 UTC
[21:26:50 INF] Creating partition events_2026_09_14
[21:26:50 INF] Partition events_2026_09_14 created successfully
[21:26:50 INF] Partition job finished with status 0k

=== Повторная проверка: POST /api/partitions/check
tasks      0k      missing: -
lab03.events 0k      missing: -   alert: recovery sent

=== GET /health/partitions
HTTP 200 Healthy: tasks = 0k; lab03.events = 0k

=== Состояние алертов (partition_alert_state)
lab03.events: 0k, изменено 21:26:53, оповещение 21:26:53
tasks: 0k, изменено 21:18:16, оповещение -
> █

```

Рисунок 2 – Восстановление: job создаёт партицию, проверка возвращает ОК и отправляет recovery

1 Партиционирование учебных таблиц

1.1 RANGE-партиционирование по дате

Таблица `events` и три дневные партии созданы по методичке в схеме `lab03`. В таблицу загружено 1 500 000 событий за 3 дня (2,5 с):

Таблица 3 – Распределение событий по партициям

Партиция	Границы	Строк
<code>events_2026_09_09</code>	<code>['2026-09-09', '2026-09-10')</code>	499 953
<code>events_2026_09_10</code>	<code>['2026-09-10', '2026-09-11')</code>	499 429
<code>events_2026_09_11</code>	<code>['2026-09-11', '2026-09-12')</code>	500 618

Границы PostgreSQL хранит в каталоге (`pg_class.relpartbound`), их можно прочесть функцией `pg_get_expr`. Этим же запросом пользуется `job` из части 10:

Листинг 1 – Результат запроса (RANGE по дате)

partition_name	bounds
events_2026_09_09	FOR VALUES FROM ('2026-09-09 00:00:00') TO ('2026-09-10 00:00:00')
events_2026_09_10	FOR VALUES FROM ('2026-09-10 00:00:00') TO ('2026-09-11 00:00:00')
events_2026_09_11	FOR VALUES FROM ('2026-09-11 00:00:00') TO ('2026-09-12 00:00:00')

Ответы на вопросы 1–4 (проверено вставкой граничных значений):

1) Запись с `created_at = '2026-09-10 12:00:00'` попала в `events_2026_09_10`: значение лежит внутри диапазона `['2026-09-10', '2026-09-11')`.

2) Запись с `created_at = '2026-09-11 00:00:00'` попала в `events_2026_09_11`: нижняя граница `FROM` включается в диапазон.

3) Запись за 2026-09-12 не вставилась: `ERROR: no partition of relation "events" found for row (SQLSTATE 23514)`. Подходящей партии нет, а `DEFAULT`-партии тоже нет, поэтому PostgreSQL отказывает в `INSERT`. Для приложения это ошибка запроса и потерянные данные, поэтому партии нужно создавать заранее (части 10–11).

4) Диапазон партиции полуоткрытый: [FROM, TO). Верхняя граница не включается, чтобы соседние партиции стыковались без пересечений и без щелей: TO одной партиции равно FROM следующей. Если бы граница включалась, значение 2026-09-11 00:00:00 принадлежало бы двум партициям. Писать «до конца дня» как '2026-09-10 23:59:59.999999' тоже плохо: такая граница зависит от точности типа и легко оставляет щель.

1.2 Partition pruning

1.2.1 Запрос с условием по ключу партиционирования

Листинг 2 – План запроса за 10 сентября

```
Finalize Aggregate (actual time=29.283..32.616 rows=1 loops=1)
  Buffers: shared hit=4726
  -> Gather (actual time=29.083..32.606 rows=3 loops=1)
    Workers Launched: 2
    -> Partial Aggregate (actual time=26.217..26.218 rows=1 loops=3)
      -> Parallel Seq Scan on events_2026_09_10 events (actual rows=166477 loops=3)
        Filter: ((created_at >= '2026-09-10 00:00:00') AND (created_at < '2026-09-
11 00:00:00'))
Planning Time: 0.075 ms
Execution Time: 32.703 ms
```

– **Сколько партиций проверил PostgreSQL.** Планировщик сравнил условие с границами всех трёх партиций, но в план попала только одна. Остальные две не читались совсем.

– **Какая партиция нужна.** events_2026_09_10: все строки за 10 сентября лежат в ней.

– **Удалось ли исключить остальные.** Да. В плане нет узла Append над несколькими партициями, прочитано 4 726 буферов, это ровно одна партиция.

– **Где в плане видно pruning.** В самом составе плана: сканируется одна партиция events_2026_09_10 вместо трёх. Это отсечение при планировании: границы запроса — константы. Если граница вычисляется при выполнении (now()), отсечение происходит на старте исполнения, и в плане появляется строка Subplans Removed: 1:

Листинг 3 – Отсечение при выполнении: граница вычисляется от now()

```
-> Parallel Append (actual time=0.012..68.215 rows=187938 loops=3)
```

```

Subplans Removed: 1
-> Parallel Seq Scan on events_2026_09_11 events_2 ...
-> Parallel Seq Scan on events_2026_09_10 events_1 ...
    Filter: (created_at >= ((now())::timestamp without time zone - '1 day'::interval))

```

Для сравнения тот же запрос выполнен на обычной таблице `events_flat` с теми же 1,5 млн строк:

Таблица 4 – Запрос за сутки: партиции против обычной таблицы

Вариант	Партиций в плане	Прочитано буферов	Rows Removed by Filter	Время
Партиции по дням	1 из 3	4 726	0	32,7 мс
Обычная таблица	—	14 208	1 000 572	50,2 мс

Таблица без партиций читает все 1,5 млн строк и отбрасывает две трети фильтром. Партиционированная таблица читает только нужную треть.

1.2.2 Запрос без ключа партиционирования

Листинг 4 – План запроса `event_type = 'click'`

```

Finalize Aggregate (actual time=37.997..42.032 rows=1 loops=1)
  Buffers: shared hit=14195
  -> Parallel Append (actual time=0.020..33.570 rows=10012 loops=3)
    -> Parallel Seq Scan on events_2026_09_11 events_3
        Filter: ((event_type)::text = 'click'::text)
        Rows Removed by Filter: 163584
    -> Parallel Seq Scan on events_2026_09_09 events_1
        Rows Removed by Filter: 244980
    -> Parallel Seq Scan on events_2026_09_10 events_2
        Rows Removed by Filter: 489256
Planning Time: 0.098 ms
Execution Time: 42.090 ms

```

Почему второй запрос обращается ко всем партициям. Условие `event_type = 'click'` не содержит ключа партиционирования `created_at`. Партиции разделены только по дате, поэтому PostgreSQL не знает, в каких из них есть клики: клик мог произойти в любой день. Отсечь нечего, и узел Append читает все три партиции (14 195 буферов, 1,47 млн строк отброшено фильтром).

1.3 RANGE-партиционирование по числовому значению

Таблица `products` разбита по цене на `products_cheap` [0, 100), `products_medium` [100, 1000) и `products_expensive` [1000, MAXVALUE). Граничные цены легли так:

Таблица 5 – Граничные значения цены

Товар	Цена	Партиция
Карандаш	15	products_cheap
Тетрадь	99,99	products_cheap
Рюкзак	100	products_medium
Наушники	999,99	products_medium
Монитор	1000	products_expensive
Ноутбук	85 000	products_expensive

После загрузки 300 000 товаров с ценой от 1 до 5 000: cheap — 5 924, medium — 53 948, expensive — 240 134. Товар с отрицательной ценой не вставился (no partition of relation "products" found for row): нижняя граница первой партии — 0.

Какие партии нужно просмотреть для `price >= 100 AND price < 500` и почему. Только products_medium: диапазон [100, 500) целиком внутри её границ [100, 1000) и не пересекается с остальными. План это подтверждает — один Seq Scan on products_medium, 448 буферов, 7,3 мс. Если диапазон задевает две партии, например `price >= 50 AND price < 150`, в плане появляются обе: Append над products_cheap и products_medium.

1.4 LIST-партиционирование

Клиенты разбиты по типу: customers_b2c (209 933), customers_b2b (75 001), customers_enterprise (15 066).

Листинг 5 – План запроса `customer_type = 'B2B'`

```
Seq Scan on customers_b2b customers (actual time=0.011..9.403 rows=75001 loops=1)
  Filter: ((customer_type)::text = 'B2B'::text)
  Buffers: shared hit=601
Planning Time: 0.138 ms
Execution Time: 12.543 ms
```

– **Какая партиция используется.** Только customers_b2b.

– **Почему PostgreSQL не обращается к другим.** Каждая партиция принимает только значения из своего списка FOR VALUES IN (...). Планировщик сравнивает константу 'B2B' со списками и знает, что в

customers_b2c и customers_enterprise таких строк быть не может. Фильтр в плане ничего не отбрасывает: в партии все строки — B2B.

– **Отличие от RANGE.** RANGE делит упорядоченное множество значений на интервалы, и отсечение работает для сравнений <, >, BETWEEN. LIST перечисляет конкретные значения без порядка: отсечение работает для = и IN (запрос IN ('B2B', 'Enterprise') читает две партии). Новое значение, которого нет ни в одном списке, требует новой партии или DEFAULT.

1.5 Неизвестное значение и DEFAULT-партия

Вставка клиента с типом VIP без DEFAULT-партии завершилась ошибкой no partition of relation "customers" found for row. После CREATE TABLE customers_default PARTITION OF customers DEFAULT та же вставка прошла, строка оказалась в customers_default.

Ответы:

1) **Зачем нужна DEFAULT partition.** В неё попадают строки, значение ключа которых не подходит ни одной другой партии. Без неё такая вставка завершается ошибкой.

2) **Чем она полезна.** Это страховка от неожиданных данных: появился новый тип клиента — приложение продолжает работать, данные не теряются, партию для нового значения можно создать позже.

3) **Какие проблемы возникают, если постоянно складывать новые значения в DEFAULT.** В эксперименте проявились все три:

– **Нельзя создать партию для значения, которое уже лежит в DEFAULT.** CREATE TABLE customers_vip PARTITION OF customers FOR VALUES IN ('VIP') упал с ошибкой updated partition constraint for default partition "customers_default" would be violated by some row. Пришлось в одной транзакции отсоединить DEFAULT, создать партию, перенести строки и присоединить DEFAULT обратно.

– **DEFAULT** читается запросами по любому значению не из списков. Запрос `customer_type = 'Premium'` сканирует `customers_default`. Когда в **DEFAULT** копяты разные значения, она превращается в обычную большую неразделённую таблицу, и пользы от партиционирования для этих значений нет.

– **DEFAULT** скрывает проблему. Ошибки нет, поэтому никто не узнаёт, что данные лежат не там. При создании каждой новой партиции PostgreSQL сканирует **DEFAULT** под блокировкой, чтобы проверить, что в ней нет строк для нового значения, — чем она больше, тем дольше блокировка.

По этой же причине в таблице `tasks` (часть 12) **DEFAULT**-партиции нет: отсутствие будущей партиции должно обнаруживаться проверкой, а не маскироваться.

1.6 HASH-партиционирование

В `user_events` с 4 **HASH**-партициями по `user_id` загружен 1 000 000 событий 100 000 пользователей за 3 года:

Таблица 6 – Распределение по **HASH**-партициям

Партиция	Строк	Доля	Пользователей
<code>user_events_0</code>	251 508	25,15 %	25 124
<code>user_events_1</code>	250 013	25,00 %	24 977
<code>user_events_2</code>	249 226	24,92 %	24 970
<code>user_events_3</code>	249 253	24,93 %	24 924

Ответы:

1) **Насколько равномерно распределились данные.** Почти идеально: отклонение от 25 % не больше 0,15 процентного пункта. Равномерность держится, пока пользователей много и ни один не даёт заметную долю строк.

2) **Чем полезен HASH.** Он делит большую таблицу на части одинакового размера, когда естественных диапазонов или категорий нет. Каждая партиция и её индексы меньше, `VACUUM` и `ANALYZE` идут по частям.

Запрос по конкретному пользователю читает одну партицию: `user_id = 42`
 — только `user_events_2`.

3) **Чем HASH отличается от RANGE.** RANGE кладёт рядом соседние значения ключа, поэтому отсекает партиции для диапазонов и позволяет удалять старые данные целыми партициями. HASH раскладывает значения по остатку от хэша: отсечение работает только для равенства, а соседние значения оказываются в разных партициях. Число HASH-партиций задаётся модулем при создании, и поменять его без перезаписи данных нельзя.

4) **Почему HASH плохо подходит для «удалить данные старше 3 лет».** Старые события равномерно размазаны по всем партициям. Запрос за период читает все 4 партиции, а удаление превращается в DELETE по всем партициям: здесь 27 713 строк за 57 мс, а на миллиардах строк это часы, огромный объём WAL и раздувание таблиц до следующего VACUUM. В RANGE по дате те же данные удаляются мгновенно: DETACH и DROP старой партиции.

1.7 Выбор стратегии партиционирования

Таблица 7 – Выбор стратегии для сценариев

Сценарий	Стратегия и ключ	Обоснование
А. Миллионы событий в день, удаление данных старше 3 лет	RANGE по <code>created_at</code> , партиция на день или месяц	Старые данные удаляются DROP/DETACH целой партиции мгновенно и без раздувания. Запросы за период читают только свои партиции
В. Три типа пользователей, запросы фильтруют по типу	LIST по <code>customer_type</code>	Значений мало, они известны заранее и не упорядочены. Запрос <code>c =</code> читает одну партицию
С. Равномерно распределить по <code>user_id</code>	HASH по <code>user_id</code>	Естественных диапазонов нет, HASH даёт партиции одинакового размера, запрос по пользователю читает одну партицию
Д. Заказы, аналитика по <code>created_at BETWEEN ...</code>	RANGE по <code>created_at</code>	Диапазон по ключу отсекает все партиции вне периода. Каждая партиция — законченный период
Е. Платежи по странам EE, LV, LT, FI, SE	LIST по коду страны	Набор стран дискретный и заранее известен. Мелкие страны можно объединить в одну партицию: <code>FOR VALUES IN ('EE', 'LV',</code>

Сценарий	Стратегия и ключ	Обоснование
		'LT'). DEFAULT — только вместе с контролем её размера

1.8 Партиционирование и индексы

Индекс `idx_events_user_id` создан на родительской таблице, PostgreSQL сам создал по индексу в каждой партиции:

Таблица 8 – Индексы партиций

Индекс	Размер
<code>events_2026_09_09_user_id_idx</code>	5 784 кБ
<code>events_2026_09_10_user_id_idx</code>	5 776 кБ
<code>events_2026_09_11_user_id_idx</code>	5 784 кБ

Листинг 6 – План запроса за сутки по `user_id`

```
Bitmap Heap Scan on events_2026_09_10 events (actual time=0.018..0.021 rows=5 loops=1)
  Recheck Cond: (user_id = 12345)
  Filter: ((created_at >= '2026-09-10 00:00:00') AND (created_at < '2026-09-11 00:00:00'))
  Buffers: shared hit=8
-> Bitmap Index Scan on events_2026_09_10_user_id_idx (actual rows=5 loops=1)
    Index Cond: (user_id = 12345)
Planning Time: 0.050 ms
Execution Time: 0.057 ms
```

1) **Какие партиции отфильтрованы.** `events_2026_09_09` и `events_2026_09_11`. В плане только `events_2026_09_10`.

2) **Используется ли индекс.** Да: `Bitmap Index Scan on events_2026_09_10_user_id_idx`, прочитано 8 буферов вместо 4 726 при полном чтении партиции.

3) **На каком уровне существует индекс.** Индекс на родителе — партиционированный: он не хранит данных, а описывает, что у каждой партиции должен быть такой индекс. Реальные B-tree — локальные индексы партиций, по одному на партицию. Новые партиции получают индекс автоматически, в том числе партиции, которые создаёт job. Глобального индекса по всей таблице в PostgreSQL нет.

4) **Почему комбинация эффективнее одного механизма.**

Таблица 9 – Поиск по user_id: партиции, индекс и их комбинация

Вариант	Что читается	Буферов	Время
Сутки + user_id, партиции + индекс	1 индекс, 1 партиция	8	0,057 мс
Только user_id, партиции + индекс	3 индекса, 3 партиции	25	0,117 мс
Сутки + user_id, обычная таблица + индекс	1 большой индекс, 16 строк за 3 дня, 11 отброшено	19	0,049 мс
Сутки, партиции без индекса (часть 2)	вся партиция	4 726	32,7 мс

Партиционирование отвечает на вопрос «в какой части данных искать», индекс — «как найти строки внутри этой части». Одно отсечение без индекса всё равно читает целую партицию (500 тыс. строк). Индекс без партиций работает, но растёт вместе со всей таблицей и возвращает строки за все дни, которые потом отбрасываются фильтром (11 из 16). Вместе они читают маленький индекс одной партиции и только нужные строки. На 1,5 млн строк разница в микросекундах. На сотнях миллионов строк индекс одной партиции помещается в память, а общий — нет.

1.9 Когда партиционирование не помогает

Таблица 10 – Запрос по event_type с индексом и без

Вариант	Партиций	Сканирование	Буферов	Rows Removed	Время
Партиции, без индекса	3 из 3	Parallel Seq Scan	14 195	1 469 968	41,6 мс
Партиции + idx_events_event_type	3 из 3	Index Only Scan	38	0	7,4 мс
То же + условие на сутки	1 из 3	Bitmap Heap Scan	4 185	0	6,6 мс

Почему партиционирование само по себе не решило проблему. Условие не содержит ключа created_at, поэтому отсечения нет, и запрос читает все партиции целиком — столько же данных, сколько обычная таблица. Партиционирование уменьшает объём чтения только для запросов по ключу. Для фильтра по другой колонке нужен индекс: он превратил полное чтение 1,5 млн строк в чтение 38 страниц индекса.

Вывод. Партиционирование отвечает на одну проблему — объём данных, который нужно просмотреть для запроса по ключу, и стоимость

обслуживания большой таблицы. Индекс отвечает на другую — быстрый поиск строк по условию.

2 Автоматизация создания и контроля партиций

2.1 Автоматическое создание партиций

Задача создания партиций реализована в сервисе TaskManager и обслуживает две таблицы: учебную lab03.events (партиция на день) и таблицу сервиса tasks (партиция на месяц, часть 12). Таблицы и горизонт задаются в конфигурации:

Листинг 7 – Настройки задачи в appsettings.json

```
"Partitioning": {
  "Enabled": true,
  "JobTimeUtc": "01:00:00",
  "CheckInterval": "00:05:00",
  "Tables": [
    { "Table": "tasks", "Interval": "Month", "PeriodsAhead": 3 },
    { "Table": "lab03.events", "Interval": "Day", "PeriodsAhead": 3, "RetentionPeriods": 30 }
  ]
}
```

Таблица 11 – Компоненты задачи

Компонент	Слой	Что делает
PartitionPlanner	Application	Чистые вычисления: начало периода, имя партиции (events_2026_09_14, tasks_2026_09), список требуемых партиций, отсутствующие, устаревшие
PartitionRepository	Infrastructure (Dapper)	Читает партиции из каталога (pg_inherits + pg_get_expr(relpartbound)), создаёт CREATE TABLE IF NOT EXISTS ... PARTITION OF ... FOR VALUES FROM ... TO ..., удаляет DETACH + DROP
PartitionMaintenanceService.EnsurePartitionsAsync	Application	Сам CreatePartitionsJob
PartitionMaintenanceWorker	API (BackgroundService)	Запускает job при старте сервиса и каждую ночь в 01:00 UTC
POST /api/partitions/ensure	API, роль Admin	Ручной запуск job

Шаги job повторяют схему из методички: определить сегодняшнюю дату (UTC) → определить горизонт (текущий период и 3 вперёд) → получить

существующие партиции → найти отсутствующие → создать их. Требования выполнены так:

Таблица 12 – Требования к job

Требование	Как выполнено
Создавать недостающие партиции	Для каждого требуемого диапазона ищется партиция, которая его покрывает. Для каждого непокрытого выполняется <code>CREATE TABLE ... PARTITION OF</code>
Не создавать уже существующие	Наличие определяется по границам из каталога, а не по имени: партиция с другим именем, но с нужным диапазоном, тоже считается. Вторая защита — <code>CREATE TABLE IF NOT EXISTS</code>
Безопасна при повторном запуске	Повторный запуск находит 0 отсутствующих и ничего не создаёт (тест <code>EnsurePartitionsAsync_ShouldBeIdempotent</code>)
Логировать результат	Каждый запуск пишет в лог сервиса число существующих, требуемых и отсутствующих партиций и каждую созданную

Первый запуск после развёртывания (сегодня по UTC — 2026-09-11, у `lab03.events` были только три партиции из части 1):

Листинг 8 – Лог первого запуска job

[21:18:16 INF] Partition job started at 2026-09-11 21:18:16 UTC
[21:18:16 INF] Table tasks: existing partitions 29, required 4, missing 0
[21:18:16 INF] Table lab03.events: existing partitions 3, required 4, missing 3
[21:18:16 INF] Creating partition events_2026_09_12
[21:18:16 INF] Partition events_2026_09_12 created successfully
[21:18:16 INF] Creating partition events_2026_09_13
[21:18:16 INF] Partition events_2026_09_13 created successfully
[21:18:16 INF] Creating partition events_2026_09_14
[21:18:16 INF] Partition events_2026_09_14 created successfully
[21:18:16 INF] Partition job finished with status Ok

Запас в один период. Горизонт сдвигается в полночь, а job запускается в 01:00. Если создавать партиции ровно на 3 дня вперёд, то с 00:00 до 01:00 проверка из части 11 видела бы, что партиции «сегодня + 3» нет, и отправляла бы ложный alert. Поэтому job создаёт на один период больше, чем требует проверка: 4 дня вперёд для events и 4 месяца для tasks (тест `CheckAsync_ShouldStayOk_AfterMidnightBeforeNightlyJob`). После этого изменения перезапуск сервиса досоздал по одной партиции:

Листинг 9 – Лог запуска job с запасом

[21:22:55 INF] Table tasks: existing partitions 29, required 5, missing 1
[21:22:55 INF] Creating partition tasks_2027_01
[21:22:55 INF] Partition tasks_2027_01 created successfully
[21:22:55 INF] Table lab03.events: existing partitions 6, required 5, missing 1
[21:22:55 INF] Creating partition events_2026_09_15

Удаление старых партиций. Для `lab03.events` задано хранение 30 дней (`RetentionPeriods`): `job` отсоединяет и удаляет партиции, которые целиком старше границы, — это DROP таблицы, а не DELETE строк (часть 6). Для `tasks` хранение не задано: задачи — рабочие данные пользователей, их нельзя удалять по возрасту.

Логика покрыта модульными тестами: планировщик, разбор границ из `pg_get_expr`, политика алертов и сервис целиком на подменённых часах (`TimeProvider`) и хранилище в памяти. Всего в проекте 93 теста, все проходят.

2.2 Сбой ночной задачи: проверка и оповещение

2.2.1 Проверка PartitionHealthCheck

Проверка — метод `PartitionMaintenanceService.CheckAsync`. Он строит тот же список требуемых партиций, что и `job`, но без запаса: текущий период и 3 вперёд. Если какой-то нет, результат — **CRITICAL**:

Таблица 13 – Результат проверки при пропавшей партиции

Дата	Партиция	Есть
11 сентября	events_2026_09_11	✓
12 сентября	events_2026_09_12	✓
13 сентября	events_2026_09_13	✓
14 сентября	events_2026_09_14	✗ → CRITICAL

Проверка доступна тремя способами:

- `PartitionMaintenanceWorker` запускает её каждые 5 минут (`CheckInterval`) и отправляет уведомления;
- `POST /api/partitions/check` — ручной запуск с уведомлениями, `GET /api/partitions` — состояние без уведомлений (роль Admin);
- `GET /health/partitions` — стандартный health check ASP.NET Core для систем мониторинга: 200 Healthy или 503 Unhealthy со списком

отсутствующих партиций. Общий `/health` эту проверку не включает: пропавшая будущая партиция не делает весь сервис «нерабочим».

2.2.2 Отправка уведомления (задание 11.1)

Канал — Telegram-бот (Bot API, метод `sendMessage`), класс `TelegramAlertNotifier`. Токен бота и `chat id` берутся из локального файла `.env`: он указан в `.gitignore` и не попадает в репозиторий (шаблон — `.env.example`). `Docker Compose` передаёт их в контейнер API переменными окружения. Токен входит в URL запроса к Telegram, поэтому для этого `HttpClient` отключено логирование запросов, иначе токен попал бы в лог. Без токена сообщение пишется только в лог.

Листинг 10 – Уведомление о проблеме

```
🔔 Partition alert

Table: lab03.events

Missing partitions:
events_2026_09_14

Expected horizon: 3 days

Checked at:
2026-09-11 21:23:30 UTC
```

2.2.3 Проверка, что alert работает (задание 11.2)

Сценарий из методички выполнен скриптом `scripts/lab-03/demo-alert.ps1` на работающем сервисе (вывод — `docs/lab-03/results/alert-demo.txt`, запись — `docs/lab-03/demo.gif`):

- 1) Будущие партиции создал `job: events_2026_09_11 ... events_2026_09_15`, `GET /health/partitions` → `HTTP 200 Healthy`.
- 2) Сбой имитирован удалением партиции: `DROP TABLE lab03.events_2026_09_14`.
- 3) `GET /health/partitions` → `HTTP 503 Unhealthy: lab03.events = Critical: missing events_2026_09_14`.

4) `POST /api/partitions/check → lab03.events Critical missing: events_2026_09_14 alert: critical alert sent.`

5) Уведомление пришло в Telegram, в логе сервиса: `Alert sent to Telegram chat ... (21:23:31).`

2.2.4 Восстановление (задание 11.3)

После `alert` система восстановлена запуском `job` (`POST /api/partitions/ensure`): он нашёл отсутствующую партицию и создал её, повторная проверка вернула ОК и отправила `recovery`:

Листинг 11 – Восстановление: вывод сценария

```
=== Восстановление: CreatePartitionsJob, POST /api/partitions/ensure
lab03.events Ok          missing: events_2026_09_14   created: events_2026_09_14

=== Повторная проверка: POST /api/partitions/check
lab03.events Ok          missing: -      alert: recovery sent

=== GET /health/partitions
HTTP 200 Healthy: tasks = Ok; lab03.events = Ok
```

Листинг 12 – Уведомление о восстановлении

```
● Partition check OK

Table: lab03.events

All required partitions exist.

Checked at:
2026-09-11 21:23:34 UTC
```

2.2.5 Один alert вместо бесконечных повторов (задание 11.4)

Состояние последней проверки хранится в таблице `partition_alert_state` (миграция 013): статус, список отсутствующих партиций, время изменения и время доставленного уведомления. Решение принимает `AlertPolicy`: уведомление отправляется только при смене состояния.

Таблица 14 – Правила отправки уведомлений

Предыдущая проверка	Текущая	Действие
нет или OK	CRITICAL	отправить alert
CRITICAL, alert доставлен, тот же список	CRITICAL	ничего
CRITICAL, alert доставлен, список изменился	CRITICAL	отправить alert: это новая проблема
CRITICAL, alert не доставлен	CRITICAL	повторить отправку
CRITICAL, alert доставлен	OK	отправить recovery
нет или OK	OK	ничего

Таблица 15 – Хронология демонстрации

Время (UTC)	Состояние	Действие
21:23:30	CRITICAL	alert отправлен
21:23:32	CRITICAL	already notified — не отправлять повторно
21:23:33	—	job создал events_2026_09_14
21:23:34	OK	recovery отправлен

Сценарий из методички (08:00 CRITICAL → alert, 08:05 и 08:10 — тишина, 08:15 OK → recovery) проверяется тестом `CheckAsync_ShouldAlertOnceAndSendRecovery` на подменённых часах. Состояние хранится в базе, а не в памяти процесса: после перезапуска сервиса старая проблема не порождает новый alert. Если Telegram недоступен, состояние сохраняется без отметки о доставке, и следующая проверка отправляет alert снова (тест `CheckAsync_ShouldRetryAlertThatWasNotDelivered`).

Ограничение решения: проверку выполняет тот же сервис, что и job. Если сервис лежит целиком, молчат оба, поэтому `/health/partitions` стоит опрашивать внешним мониторингом (Prometheus, Uptime Kuma и т. п.).

3 Партиционирование базы данных сервиса TaskManager

3.1 Шаги 1–4. Таблица, ключ, стратегия и обоснование

Таблица 16 – Решение для сервиса TaskManager

Шаг	Решение
Таблица	tasks
Ключ	created_at
Стратегия	RANGE, партиция на календарный месяц (tasks_2026_09), без DEFAULT-партиции
Горизонт	текущий месяц + 3 вперёд (job создаёт + 4)

Почему tasks. Это самая большая таблица сервиса: 5 000 001 строка, 1,5 ГБ с индексами, и она растёт быстрее остальных — около 208 тысяч задач в месяц. Остальные таблицы маленькие (пользователи, проекты, теги). Связи task_tags — производная таблица: они живут и удаляются вместе с задачей.

Почему created_at. Ключ удовлетворяет всем требованиям из контрольного вопроса 7:

- он есть в самых частых запросах: список задач по умолчанию сортируется по created_at DESC, а в API добавлен фильтр по периоду создания createdFrom/createdTo;

- он не меняется: UpdateAsync не трогает CreatedAt, поэтому строка никогда не переезжает между партициями;

- он всегда заполнен (NOT NULL DEFAULT now()) и даёт партиции одинакового размера;

- он совпадает с жизненным циклом данных: новые задачи читают и меняют чаще старых.

Другие варианты отклонены. HASH по user_id не помогает запросам за период и архивированию, а запросы по пользователю уже обслуживает индекс (user_id, created_at). LIST по status — ключ меняется (задача переходит Pending → Completed и переезжает в другую партицию), а значений всего 4 с перекосом: Completed — 55 %. project_id может быть NULL, и значений много.

Почему RANGE по месяцу. RANGE — естественная стратегия для времени: запрос за период читает только свои партии. Размер партии выбран по измерениям:

Таблица 17 – Цена числа партий: 1 млн строк, запрос без ключа

Партий	Planning Time	Execution Time	Запрос по ключу
10	0,139 мс	32,1 мс	3,9 мс
100	0,642 мс	36,1 мс	0,36 мс
1 000	12,053 мс	97,2 мс	0,047 мс

Чем больше партий, тем быстрее запросы по ключу, но тем дороже все остальные: у запроса без ключа время планирования выросло в 87 раз, а выполнения — в 3 раза. При партии на день у `tasks` было бы около 750 партий по 7 тысяч строк, и запросы по `id` и статистика платили бы эту цену. Партия на год слишком крупная: отчёт за месяц читал бы 2,5 млн строк. Месяц даёт 12 партий в год по ~208 тысяч строк (~50 МБ).

DEFAULT-партии нет. Задачи создаются «сейчас», поэтому ключ всегда попадает в текущий месяц. Если партии нет, это сбой `job`, и его должна обнаружить проверка, а не скрыть DEFAULT (часть 5).

Какие запросы должны получить пользу. Списки задач за период `GET /api/tasks?createdFrom=...&createdTo=...` для администратора и пользователя, в том числе с фильтром по статусу, и первая страница без фильтров. Не получают пользы запросы без ключа: задача по `id` и статистика по всем задачам.

3.2 Миграция таблицы

Партиционирование выполнено миграцией Liquibase `012-partition-tasks-by-created-at.xml`. Существующую таблицу нельзя сделать партиционированной командой `ALTER`, поэтому миграция:

1) переименовывает `tasks` в `tasks_unpartitioned` и создаёт новую `tasks ... PARTITION BY RANGE (created_at);`

2) создаёт партии от самого раннего месяца с данными до текущего месяца + 3;

- 3) переносит строки (`INSERT ... SELECT`) и удаляет старую таблицу;
- 4) создаёт первичный ключ, внешние ключи и 6 индексов на родителе — PostgreSQL создаёт их копии в каждой партиции;
- 5) `ANALYZE`; блок `rollback` возвращает обычную таблицу.

На 5 млн задач и 4 млн связей миграция заняла около 3 минут. Два ограничения PostgreSQL потребовали изменений в коде:

Таблица 18 – Ограничения PostgreSQL и изменения в сервисе

Ограничение	Изменение
Первичный и уникальные ключи партиционированной таблицы обязаны включать ключ партиционирования	<code>tasks_pkey = (id, created_at)</code> . Уникальность одного <code>id</code> теперь обеспечивает приложение: <code>Guid.NewGuid()</code>
Внешний ключ может ссылаться только на уникальный ключ целиком	В <code>task_tags</code> добавлена колонка <code>task_created_at</code> , внешний ключ <code>(task_id, task_created_at) → tasks (id, created_at)</code> . В EF Core — <code>HasPrincipalKey(t => new { t.Id, t.CreatedAt })</code> , вставка через <code>Dapper</code> берёт <code>created_at</code> из <code>tasks</code>

3.3 Шаг 5. Партиции

Миграция создала 29 партиций (`tasks_2024_08 ... tasks_2026_12`), `job` досоздал `tasks_2027_01`. Строки распределились равномерно: полный месяц — 207 735 ... 208 918 задач, первый месяц — 139 655, текущий — 68 502 (месяц не закончился), будущие пусты. Размер таблицы с индексами уменьшился с 1 502 МБ до 1 414 МБ: таблица переписана заново, без «мёртвых» строк от прошлых экспериментов.

3.4 Шаг 6. Запросы API и `partition pruning`

Каждый запрос выполнен на той же базе дважды — до миграции 012 (обычная таблица) и после неё. Запросы Q1–Q8 повторяют часть SQL, которую EF Core строит по таблице `tasks` (`scripts/lab-03/10-service-queries.sql`):

Таблица 19 – Запросы к tasks до и после партиционирования

Запрос	Endpoint	Партиций в плане / выполнено	Буферов до → после	Время до → после
Q1 COUNT за месяц	GET /api/tasks?createdFrom&createdTo	1 / 1 из 29	137 572 → 3 655	36,6 → 32,0 мс
Q2 страница за месяц	то же	1 / 1	13 → 13	0,043 → 0,035 мс
Q3 COUNT за неделю + статус	GET /api/tasks?status&createdFrom&createdTo	1 / 1	20 982 → 1 217	297,0 → 3,8 мс
Q4 COUNT пользователя за месяц	GET /api/tasks?createdFrom&createdTo (User)	1 / 1	18 → 15	0,498 → 0,302 мс
Q5 страница пользователя	то же	1 / 1	13 → 13	0,036 → 0,032 мс
Q6 первая страница без фильтров	GET /api/tasks	29 / 4	13 → 15	0,056 → 0,208 мс
Q7 задача по id	GET /api/tasks/{id}	29 / 26	4 → 76	0,024 → 0,558 мс
Q8 статистика по статусам	GET /api/tasks/stats	29 / 29	87 919 → 87 933	409,8 → 593,3 мс

Затем в логе PostgreSQL снят полный SQL, который EF Core отправляет для endpoint'ов (capture-api-sql.sh), и на нём выполнен EXPLAIN (ANALYZE, BUFFERS) (11-api-queries.sql; к этому моменту job создал 30-ю партицию):

Таблица 20 – Реальные запросы API на партиционированной tasks

Запрос API (EF Core)	Партиций в плане / выполнено	Узел сканирования	Буферов	Время
A1 GET /api/tasks?createdFrom=2026-08-01&createdTo=2026-08-31T23:59:59, COUNT	1 / 1 из 30	Seq Scan on tasks_2026_08	3 655	17,4 мс
A1 то же, страница с JOIN проектов, пользователей и тегов	1 / 1	Index Scan on tasks_2026_08	117	0,395 мс
A2 GET /api/tasks?status=Pending&createdFrom=...&createdTo=..., COUNT	1 / 1	Bitmap Heap Scan on tasks_2026_09	1 217	4,1 мс
A3 то же для пользователя, страница	1 / 1	Index Scan on tasks_2026_08	81	0,372 мс
A4 GET /api/tasks/{id}	30 / 26	Index Scan по PK каждой партиции	90	0,875 мс
A5 GET /api/tasks/stats	30 / 30	Seq Scan всех партиций	87 933	603,8 мс
A6 GET /api/tasks, первая страница	30 / 5	Index Scan, остальные never executed	112	0,713 мс

Partition pruning используют запросы A1–A3: условие на created_at задано константами, и в плане остаётся одна партиция из 30:

Листинг 13 – A1: в плане только партиция за август

```
-> Index Scan using tasks_2026_08_created_at_due_date_idx on tasks_2026_08 t  
    (actual time=0.017..0.022 rows=10 loops=1)  
    Index Cond: ((created_at >= '2026-08-01 00:00:00+00') AND (created_at <= '2026-08-31  
23:59:59+00'))
```

Листинг 14 – A2: неделя сентября и статус — одна партиция и её индекс по статусу

```
-> Bitmap Heap Scan on tasks_2026_09 t (actual time=0.603..3.575 rows=9844 loops=1)  
    -> Bitmap Index Scan on tasks_2026_09_status_idx (actual rows=13853 loops=1)  
    Execution Time: 4.072 ms
```

Разбор результатов.

– **Q3/A2 ускорился в 78 раз.** До партиционирования PostgreSQL объединял два индекса (BitmapAnd): диапазон дат дал 48 557 строк, а status = 0 — 1 000 904 строки. Только построение битовой карты по статусу заняло 200 мс, затем читались 20 007 страниц, разбросанных по всей таблице. После партиционирования запрос работает внутри tasks_2026_09: там 68 502 строки, индекс по статусу даёт 13 853, и читается 1 217 страниц одной небольшой партиции.

– **Q1 почти не изменился по времени.** На обычной таблице это Index Only Scan по диапазону индекса (137 тысяч обращений к карте видимости), на партиционированной — полное чтение одной партиции: 3 655 страниц. Оба способа читают только нужный месяц. При повторном запуске через API получилось 17 мс.

– **Страницы списков (Q2, Q4, Q5, A1, A3) и так брали 10 строк из индекса за доли миллисекунды.** Партиционирование их не ухудшило, время планирования выросло на десятые доли миллисекунды.

– **Q6/A6 — не отсечение, а упорядоченный Append.** Партиции идут в порядке created_at, поэтому PostgreSQL читает их с самой новой. Первые 10 строк нашлись в tasks_2026_09, остальные 25 партиций помечены never executed.

– **Запросы без ключа стали дороже.** GET /api/tasks/{id} проверяет индекс первичного ключа каждой партиции, пока не найдёт строку: 26

проверок вместо одной, $0,02 \rightarrow 0,56$ мс. Это всё ещё меньше миллисекунды, но время растёт с числом партиций. Если передавать в API и `created_at`, запрос уйдёт в одну партицию. Статистика по всем задачам читает всё и стала медленнее на 45 % из-за накладных расходов Append над 30 партициями. Такой запрос нужно кэшировать или считать заранее (материализованное представление из ЛР № 2).

3.5 Шаги 7–9. Автоматизация, контроль и alert

Для `tasks` работает тот же механизм, что и для `lab03.events` (части 10–11):

- **шаг 7:** `job` создаёт партиции на текущий месяц + 4 вперёд при старте сервиса и каждую ночь в 01:00 UTC; хранение не ограничено;
- **шаг 8:** проверка требует текущий месяц + 3 и видна в `GET /health/partitions (tasks = Ok)` и `GET /api/partitions`;
- **шаг 9:** при пропаже партиции приходит такой же alert в Telegram с `Table: tasks` и `Expected horizon: 3 months`.

Сценарий для защиты по методичке:

Таблица 21 – Сценарий защиты и где он показан

Шаг	Где показано
Большая таблица	<code>tasks</code> : 5 млн строк, 1,5 ГБ, +208 тыс. в месяц (шаги 1–4)
Выбор ключа и стратегии	<code>created_at</code> , RANGE по месяцу; обоснование и замер цены числа партиций
Партиции	миграция 012: 30 партиций, распределение по месяцам (шаг 5)
Partition pruning	EXPLAIN запросов API: 1 партиция из 30 (шаг 6)
Автоматическое создание	лог <code>job: tasks_2027_01, events_2026_09_12 ... 15</code> (часть 10)
Проверка → сбой → ALERT	<code>DROP TABLE events_2026_09_14</code> → 503 Unhealthy → alert в Telegram (часть 11, рисунок 1)
Восстановление → OK	<code>job</code> создаёт партицию → recovery в Telegram → 200 Healthy (часть 11, рисунок 2)

4 Ответы на контрольные вопросы

1) **Что такое partitioning?** Разделение одной логической таблицы на несколько физических таблиц (партиций) по значению ключа. Приложение работает с родительской таблицей, а PostgreSQL сам кладёт строку в нужную партицию при вставке и выбирает нужные партиции при чтении.

2) **Чем partitioning отличается от индекса?** Индекс — дополнительная структура для быстрого поиска строк по условию. Как хранится таблица, он не меняет. Партиционирование меняет само хранение: данные разложены по отдельным таблицам. Это уменьшает объём чтения для запросов по ключу и позволяет обслуживать данные частями: удалить партицию, пройти `VACUUM` по одной партиции. Механизмы дополняют друг друга (часть 8).

3) **Какие стратегии partitioning поддерживает PostgreSQL?** RANGE (диапазоны), LIST (списки значений) и HASH (остаток от хэша). Для RANGE и LIST есть DEFAULT-партиция. Ключ может состоять из нескольких колонок или выражения, партиции можно партиционировать дальше (многоуровневое партиционирование).

4) **Когда использовать RANGE?** Когда ключ упорядочен, а запросы и жизненный цикл данных идут диапазонами: события, логи, заказы, платежи по дате. Архив и удаление старых данных делаются целыми партициями.

5) **Когда использовать LIST?** Когда ключ принимает небольшой заранее известный набор дискретных значений, а запросы фильтруют по равенству: тип клиента, страна, регион, `tenant`.

6) **Когда использовать HASH?** Когда нужно поделить большую таблицу на равные части, а естественных диапазонов и категорий нет, и запросы ищут по равенству ключа (`user_id = ...`). Для запросов по диапазону и для удаления старых данных HASH не подходит.

7) **Как выбрать partition key?** Ключ должен:

- стоять в условиях самых частых и тяжёлых запросов, иначе отсечения не будет;
- не меняться — при изменении ключа строка переезжает в другую партицию;
- быть NOT NULL и давать партиции сопоставимого размера;
- соответствовать жизненному циклу данных (архив по времени).

Кроме того, ключ обязан входить в первичный и уникальные ключи. Для `tasks` выбран `created_at` (часть 12).

8) Что такое `partition pruning`? Исключение из запроса партиций, в которых по их границам не может быть подходящих строк. При константах в условии это происходит при планировании: в плане остаются только нужные партиции (часть 2: 1 из 3). Для параметров, `now()` и подзапросов — при выполнении: `Subplans Removed` или `never executed` в плане.

9) Почему `pruning` может не сработать?

- в условии нет ключа партиционирования (`event_type = 'click'`);
- ключ обернут в функцию или приведение: `date(created_at) = ..., created_at::date;`
- условие по ключу стоит в OR с другими условиями;
- типы в сравнении не совпадают;
- отключён `enable_partition_pruning`.

Кроме того, DEFAULT-партиция остаётся кандидатом для любых значений, которых нет в списках.

10) Можно ли использовать индексы вместе с `partitioning`? Да. Индекс, созданный на родителе, создаётся в каждой партиции, в том числе в будущих, которые создаст `job`. Уникальные индексы должны включать ключ партиционирования, глобального индекса на всю таблицу нет. Лучший результат даёт комбинация: `pruning` выбирает партицию, индекс находит строки в ней (часть 8: 8 буферов вместо 4 726).

11) **Что произойдёт, если подходящей партиции нет?** INSERT завершится ошибкой `no partition of relation "... found for row (SQLSTATE 23514)`, и данные не сохранятся. Если есть DEFAULT-партиция, строка попадёт в неё.

12) **Зачем создавать будущие партиции заранее?** Данные за новый день начинают поступать в 00:00:00, и партиция к этому моменту уже должна существовать. Создание партиции ненадолго блокирует родительскую таблицу, поэтому его лучше делать заранее и в тихое время. Запас в несколько периодов даёт время заметить сбой job и исправить его до того, как начнут падать вставки: при горизонте 3 дня на реакцию есть 3 дня.

13) **Почему создание партиций должно быть автоматизировано?** Календарь не останавливается, а человек забывает, болеет или уходит в отпуск. Ручная операция каждую ночь рано или поздно будет пропущена. Автоматическая job одинакова при каждом запуске, безопасна при повторе и пишет результат в лог. Но автоматизация тоже ломается, поэтому нужны ещё проверка и alert (часть 11).

14) **Чем логирование ошибки отличается от alerting?** Лог сохраняет факт для последующего разбора, но сам никого не зовёт: пока лог не прочитали, проблемы как бы нет. Alert активно сообщает ответственному человеку, что нужно действовать сейчас. У нас job пишет в лог каждый запуск, а alert отправляется только когда состояние становится CRITICAL или восстанавливается.

15) **Что такое recovery alert?** Сообщение о том, что проблема, о которой сообщали раньше, устранена. Оно закрывает инцидент: люди знают, что действовать больше не нужно. Без него нельзя отличить «проблему исправили» от «проверка сломалась и молчит».

16) **Почему нельзя бесконечно отправлять одинаковый alert?** Одинаковые сообщения каждые 5 минут быстро начинают игнорировать или отключают канал совсем — это «усталость от алертов». Тогда новая, настоящая проблема тонет в потоке. Кроме того, бесконечные сообщения

упираются в лимиты канала. Поэтому уведомление отправляется при смене состояния (часть 11, задание 11.4).

17) **Всегда ли partitioning ускоряет запросы?** Нет. Он ускоряет запросы по ключу: в сервисе запрос за неделю со статусом ускорился с 297 до 3,8 мс. Запросы без ключа он не ускоряет и может замедлить: поиск задачи по `id` — 0,02 → 0,56 мс, статистика по всем задачам — 410 → 593 мс. С ростом числа партиций растёт время планирования (1 000 партиций — 12 мс). Кроме того, он добавляет ограничения: ключ входит в первичный ключ, внешние ключи на таблицу усложняются.

18) **Какие проблемы возникают при неправильном выборе размера партиций?** Слишком мелкие партиции (их слишком много) увеличивают время планирования и выполнения запросов без ключа — на 1 000 партиций 12 мс планирования и 97 мс выполнения против 0,14 и 32 мс на 10. Растут память под метаданные, число блокировок и файлов. Слишком крупные партиции почти не дают отсечения: запрос читает большую партицию целиком, `VACUUM` и перестроение индексов тяжёлые, старые данные удаляются слишком большими порциями. Неравномерные партиции создают «горячую» партицию, которая растёт и тормозит. Размер нужно выбирать так, чтобы типичный запрос читал одну или несколько партиций, а всего партиций было не больше сотен.

ЗАКЛЮЧЕНИЕ

В работе на учебных таблицах изучены три стратегии партиционирования PostgreSQL. RANGE по дате и по цене отсекает партии для запросов по диапазону: выборка за сутки читает одну треть данных, 4 726 буферов вместо 14 208. LIST отсекает по равенству, а DEFAULT-партия страхует от неизвестных значений, но мешает создавать новые партии и скрывает проблему. HASH распределил миллион строк по четырём партициям с отклонением не больше 0,15 процентного пункта, но плохо подходит для удаления старых данных. Измерения подтвердили, что партиционирование и индекс решают разные задачи: запрос по колонке вне ключа читает все партии, пока нет индекса (41,6 → 7,4 мс).

В сервисе TaskManager реализована эксплуатационная часть:

- **CreatePartitionsJob** — идемпотентная задача, которая по конфигурации создаёт недостающие партии на горизонт с запасом в один период и удаляет устаревшие;
- **PartitionHealthCheck** — проверка с HTTP-эндпоинтами и health check;
- **alert в Telegram** — отправляется один раз при сбое, повторно не дублируется, при восстановлении приходит recovery.

Сценарий «ночной job не отработал» проверен на работающем сервисе: удалённая партия обнаружена, alert доставлен, после восстановления пришёл recovery.

Таблица `tasks` (5 млн строк) переведена на RANGE-партии по месяцу `created_at`, для чего первичный ключ и внешний ключ связей с тегами расширены ключом партиционирования. Запросы API с периодом создания читают одну партию из 30. Самый тяжёлый из них — неделя со статусом — ускорился в 78 раз, 297 → 3,8 мс. Запросы без ключа (по `id`, общая статистика) стали дороже, это цена выбранной схемы. Главный вывод работы: партиционирование — это не одна команда `PARTITION BY`, а

жизненный цикл: выбор ключа по реальным запросам, создание партиций заранее, контроль их наличия и оповещение о сбое.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 PostgreSQL 16 Documentation. 5.11. Table Partitioning [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/ddl-partitioning.html> (дата обращения: 12.09.2026).
- 2 PostgreSQL 16 Documentation. CREATE TABLE [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/sql-createtable.html> (дата обращения: 12.09.2026).
- 3 PostgreSQL 16 Documentation. 14.1. Using EXPLAIN [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/using-explain.html> (дата обращения: 12.09.2026).
- 4 PostgreSQL 16 Documentation. 53.38. pg_partitioned_table [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/catalog-pg-partitioned-table.html> (дата обращения: 12.09.2026).
- 5 Health checks in ASP.NET Core [Электронный ресурс]. – URL: <https://learn.microsoft.com/aspnet/core/host-and-deploy/health-checks> (дата обращения: 12.09.2026).
- 6 Telegram Bot API [Электронный ресурс]. – URL: <https://core.telegram.org/bots/api> (дата обращения: 12.09.2026).
- 7 Лабораторная работа № 3. Партиционирование PostgreSQL: разделение больших таблиц и автоматизация создания партиций : методические указания. – Сириус : Колледж Университета «Сириус», 2026. – 18 с.
- 8 Партиционирование. Эволюция базы данных: от одной таблицы до распределённой системы : презентация к занятию. – Сириус : Колледж Университета «Сириус», 2026. – 11 с.
- 9 VHS — инструмент записи терминала [Электронный ресурс]. – URL: <https://github.com/charmbracelet/vhs> (дата обращения: 12.09.2026).

ПРИЛОЖЕНИЕ А

Миграция таблицы tasks на партиции

Миграция Liquibase из подраздела 3.2. Применяется автоматически при запуске `docker compose up`, блок `rollback` возвращает обычную таблицу.

Листинг А.1 – 012-partition-tasks-by-created-at.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
  xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
    http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.25.xsd">

  <!--
    ЛР №3 (docs/lab-03-partitioning.md, часть 12): tasks → RANGE-партиции по created_at, по
    месяцу.

    Ограничения PostgreSQL, которые определяют форму миграции:
    - первичный ключ партиционированной таблицы обязан включать ключ партиционирования:
      tasks_pkey = (id, created_at);
    - внешний ключ может ссылаться только на уникальный ключ целиком, поэтому task_tags
      хранит task_created_at и ссылается на tasks (id, created_at).

    Партиции создаются от самого раннего месяца с данными (не позже чем за 25 месяцев –
    столько покрывает генератор тестовых данных) до текущего месяца + 3 вперёд.
    Далее будущие партиции создаёт фоновая задача сервиса (PartitionMaintenanceWorker).
  -->
  <changeSet id="012-partition-tasks-by-created-at" author="taskmanager">
    <sql>ALTER TABLE task_tags DROP CONSTRAINT fk_task_tags_task_id</sql>
    <sql>ALTER TABLE tasks RENAME TO tasks_unpartitioned</sql>
    <sql>CREATE TABLE tasks (LIKE tasks_unpartitioned INCLUDING DEFAULTS) PARTITION BY
    RANGE (created_at)</sql>

    <sql splitStatements="false">
DO $$
DECLARE
  first_month TIMESTAMPTZ;
  last_month TIMESTAMPTZ;
  m TIMESTAMPTZ;
BEGIN
  PERFORM set_config('TimeZone', 'UTC', true);
  first_month := date_trunc('month', LEAST(
    COALESCE((SELECT min(created_at) FROM tasks_unpartitioned), now()),
    now() - INTERVAL '25 months'));
  last_month := date_trunc('month', now()) + INTERVAL '3 months';
  m := first_month;
  WHILE m <= last_month LOOP
    EXECUTE format('CREATE TABLE %I PARTITION OF tasks FOR VALUES FROM (%L) TO (%L)',
      'tasks_' || to_char(m, 'YYYY-MM'), m, m + INTERVAL '1 month');
    m := m + INTERVAL '1 month';
  END LOOP;
END $$
    </sql>

    <!-- данные переносятся до создания индексов: так вставка быстрее -->
    <sql>INSERT INTO tasks SELECT * FROM tasks_unpartitioned</sql>
    <sql>DROP TABLE tasks_unpartitioned</sql>

    <sql>ALTER TABLE tasks ADD CONSTRAINT tasks_pkey PRIMARY KEY (id, created_at)</sql>
    <sql>ALTER TABLE tasks ADD CONSTRAINT fk_tasks_user_id FOREIGN KEY (user_id) REFERENCES
    users (id) ON DELETE CASCADE</sql>
    <sql>ALTER TABLE tasks ADD CONSTRAINT fk_tasks_project_id FOREIGN KEY (project_id)
    REFERENCES projects (id) ON DELETE SET NULL</sql>
    <sql>CREATE INDEX idx_tasks_user_id_created_at ON tasks (user_id, created_at
    DESC)</sql>
    <sql>CREATE INDEX idx_tasks_created_at ON tasks (created_at DESC,
```

```

due_date)</sql>
    <sql>CREATE INDEX idx_tasks_project_id ON tasks (project_id)</sql>
    <sql>CREATE INDEX idx_tasks_status ON tasks (status)</sql>
    <sql>CREATE INDEX idx_tasks_priority ON tasks (priority)</sql>
    <sql>CREATE INDEX idx_tasks_due_date ON tasks (due_date)</sql>

    <sql>ALTER TABLE task_tags ADD COLUMN task_created_at TIMESTAMPTZ</sql>
    <sql>UPDATE task_tags tt SET task_created_at = t.created_at FROM tasks t WHERE t.id =
tt.task_id</sql>
    <sql>DELETE FROM task_tags WHERE task_created_at IS NULL</sql>
    <sql>ALTER TABLE task_tags ALTER COLUMN task_created_at SET NOT NULL</sql>
    <sql>ALTER TABLE task_tags ADD CONSTRAINT fk_task_tags_task FOREIGN KEY (task_id,
task_created_at) REFERENCES tasks (id, created_at) ON DELETE CASCADE</sql>
    <sql>ANALYZE tasks</sql>
    <sql>ANALYZE task_tags</sql>

    <rollback>
        <sql>ALTER TABLE task_tags DROP CONSTRAINT fk_task_tags_task</sql>
        <sql>ALTER TABLE task_tags DROP COLUMN task_created_at</sql>
        <sql>ALTER TABLE tasks RENAME TO tasks_partitioned</sql>
        <sql>CREATE TABLE tasks (LIKE tasks_partitioned INCLUDING DEFAULTS)</sql>
        <sql>INSERT INTO tasks SELECT * FROM tasks_partitioned</sql>
        <sql>DROP TABLE tasks_partitioned</sql>
        <sql>ALTER TABLE tasks ADD CONSTRAINT tasks_pkey PRIMARY KEY (id)</sql>
        <sql>ALTER TABLE tasks ADD CONSTRAINT fk_tasks_user_id FOREIGN KEY (user_id)
REFERENCES users (id) ON DELETE CASCADE</sql>
        <sql>ALTER TABLE tasks ADD CONSTRAINT fk_tasks_project_id FOREIGN KEY (project_id)
REFERENCES projects (id) ON DELETE SET NULL</sql>
        <sql>CREATE INDEX idx_tasks_user_id_created_at ON tasks (user_id, created_at
DESC)</sql>
        <sql>CREATE INDEX idx_tasks_created_at_due_date ON tasks (created_at DESC,
due_date)</sql>
        <sql>CREATE INDEX idx_tasks_project_id ON tasks (project_id)</sql>
        <sql>CREATE INDEX idx_tasks_status ON tasks (status)</sql>
        <sql>CREATE INDEX idx_tasks_priority ON tasks (priority)</sql>
        <sql>CREATE INDEX idx_tasks_due_date ON tasks (due_date)</sql>
        <sql>ALTER TABLE task_tags ADD CONSTRAINT fk_task_tags_task_id FOREIGN KEY
(task_id) REFERENCES tasks (id) ON DELETE CASCADE</sql>
    </rollback>
</changeSet>

</databaseChangeLog>

```

ПРИЛОЖЕНИЕ Б

Задача создания партиций и проверка

Класс PartitionMaintenanceService: CreatePartitionsJob (EnsurePartitionsAsync) и PartitionHealthCheck с оповещением (CheckAsync), раздел 2.

Листинг Б.1 – PartitionMaintenanceService.cs

```
using Microsoft.Extensions.Logging;
using TaskManager.Application.Interfaces;
using TaskManager.Application.Partitioning;

namespace TaskManager.Application.Services;

/// <summary>
/// CreatePartitionsJob and PartitionHealthCheck for time-partitioned tables.
/// </summary>
public class PartitionMaintenanceService : IPartitionMaintenanceService
{
    // The job creates one period more than the check requires: the horizon moves at midnight,
    // the nightly job runs later (JobTimeUtc), and in between the check must stay OK.
    private const int ReservePeriods = 1;

    private readonly IPartitionRepository _repository;
    private readonly IAlertNotifier _notifier;
    private readonly PartitioningOptions _options;
    private readonly TimeProvider _time;
    private readonly ILogger<PartitionMaintenanceService> _logger;

    /// <summary>
    /// Initializes a new instance of the <see cref="PartitionMaintenanceService"/> class.
    /// </summary>
    public PartitionMaintenanceService(
        IPartitionRepository repository,
        IAlertNotifier notifier,
        PartitioningOptions options,
        TimeProvider time,
        ILogger<PartitionMaintenanceService> logger)
    {
        _repository = repository;
        _notifier = notifier;
        _options = options;
        _time = time;
        _logger = logger;
    }

    /// <inheritdoc/>
    public async Task<PartitionRunResult> EnsurePartitionsAsync(CancellationToken
        cancellationToken = default)
    {
        var now = _time.GetUtcNow().UtcDateTime;
        var result = new PartitionRunResult { CheckedAt = now };
        _logger.LogInformation("Partition job started at {Now:yyyy-MM-dd HH:mm:ss} UTC", now);

        foreach (var table in _options.Tables)
        {
            var report = new PartitionTableReport { Table = table.Table };
            result.Tables.Add(report);

            try
            {
                var plan = await PlanAsync(table, table.PeriodsAhead + ReservePeriods, now,
                    report, cancellationToken);
                if (plan is null)
            }
        }
    }
}
```

```

        {
            continue;
        }

        foreach (var partition in plan.Value.Missing)
        {
            _logger.LogInformation("Creating partition {Partition}", partition.Name);
            await _repository.CreatePartitionAsync(table.Table, partition,
cancellationTokens);
            report.CreatedPartitions.Add(partition.Name);
            _logger.LogInformation("Partition {Partition} created successfully",
partition.Name);
        }

        if (table.RetentionPeriods is { } retention)
        {
            foreach (var expired in PartitionPlanner.Expired(plan.Value.Existing, now,
table.Interval, retention))
            {
                await _repository.DropPartitionAsync(table.Table, expired.Name,
cancellationTokens);
                report.DroppedPartitions.Add(expired.Name);
                _logger.LogInformation("Partition {Partition} is older than {Retention}
periods and was dropped",
                    expired.Name, retention);
            }
        }

        report.Status = PartitionStatus.Ok;
    }
    catch (Exception ex) when (ex is not OperationCanceledException)
    {
        report.Status = PartitionStatus.Critical;
        report.Error = ex.Message;
        _logger.LogError(ex, "Partition job failed for table {Table}", table.Table);
    }
}

result.Status = Worst(result.Tables);
_logger.LogInformation("Partition job finished with status {Status}", result.Status);
return result;
}

/// <inheritdoc>
public async Task<PartitionRunResult> CheckAsync(bool notify, CancellationToken
cancellationTokens = default)
{
    var now = _time.GetUtcNow().UtcDateTime;
    var result = new PartitionRunResult { CheckedAt = now };

    foreach (var table in _options.Tables)
    {
        var report = new PartitionTableReport { Table = table.Table };
        result.Tables.Add(report);

        try
        {
            var plan = await PlanAsync(table, table.PeriodsAhead, now, report,
cancellationTokens);
            if (plan is null)
            {
                continue;
            }

            report.Status = plan.Value.Missing.Count == 0 ? PartitionStatus.Ok :
PartitionStatus.Critical;
            if (report.Status == PartitionStatus.Critical)
            {
                _logger.LogWarning("Partition check CRITICAL for {Table}: missing
{Missing}",
                    table.Table, string.Join(", ", report.MissingPartitions));
            }
        }
    }
}

```

```

        if (notify)
        {
            report.Alert = await NotifyAsync(table, report, now, cancellationToken);
        }
    }
    catch (Exception ex) when (ex is not OperationCanceledException)
    {
        report.Status = PartitionStatus.Critical;
        report.Error = ex.Message;
        _logger.LogError(ex, "Partition check failed for table {Table}", table.Table);
    }
}

result.Status = Worst(result.Tables);
return result;
}

private async Task<IReadOnlyList<ExistingPartition> Existing,
IReadOnlyList<PartitionRange> Missing)?> PlanAsync(
    PartitionedTableOptions table, int periodsAhead, DateTime now, PartitionTableReport
report,
    CancellationTokens cancellationToken)
{
    if (!await _repository.IsPartitionedAsync(table.Table, cancellationToken))
    {
        report.Status = PartitionStatus.Skipped;
        _logger.LogWarning("Table {Table} does not exist or is not partitioned, skipped",
table.Table);
        return null;
    }

    var existing = await _repository.GetPartitionsAsync(table.Table, cancellationToken);
    var required = PartitionPlanner.RequiredPartitions(table.Table, now, table.Interval,
periodsAhead);
    var missing = PartitionPlanner.FindMissing(required, existing);

    report.ExistingPartitions = existing.Count;
    report.RequiredPartitions = required.Select(p => p.Name).ToList();
    report.MissingPartitions = missing.Select(p => p.Name).ToList();
    _logger.LogInformation(
        "Table {Table}: existing partitions {Existing}, required {Required}, missing
{Missing}",
        table.Table, existing.Count, required.Count, missing.Count);

    return (existing, missing);
}

private async Task<string?> NotifyAsync(
    PartitionedTableOptions table, PartitionTableReport report, DateTime now,
CancellationTokens cancellationToken)
{
    var missing = string.Join(", ", report.MissingPartitions);
    var previous = await _repository.GetAlertStateAsync(table.Table, cancellationToken);
    var action = AlertPolicy.Decide(previous, report.Status, missing);
    var changed = previous is null || previous.Status != report.Status ||
previous.MissingPartitions != missing;
    var changedAt = changed ? now : previous!.ChangedAt;

    if (action == AlertAction.None)
    {
        if (changed)
        {
            await _repository.SaveAlertStateAsync(
                new PartitionAlertState(table.Table, report.Status, missing, changedAt,
null), cancellationToken);
        }

        return report.Status == PartitionStatus.Critical ? "already notified" : null;
    }

    var message = action == AlertAction.SendCritical
        ? AlertMessages.Critical(table.Table, report.MissingPartitions,
PartitionPlanner.Horizon(table.Interval, table.PeriodsAhead), now)

```

```

        : AlertMessages.Recovery(table.Table, now);

        try
        {
            await _notifier.SendAsync(message, cancellationToken);
        }
        catch (Exception ex) when (ex is not OperationCanceledException)
        {
            _logger.LogError(ex, "Failed to send partition alert for {Table}", table.Table);

            // An undelivered CRITICAL is saved without NotifiedAt, so the next check sends it
            again.
            // An undelivered recovery keeps the old CRITICAL state, so the next check retries
            the recovery.
            if (action == AlertAction.SendCritical)
            {
                await _repository.SaveAlertStateAsync(
                    new PartitionAlertState(table.Table, report.Status, missing, changedAt,
                    null), cancellationToken);
            }

            return "failed";
        }

        await _repository.SaveAlertStateAsync(
            new PartitionAlertState(table.Table, report.Status, missing, changedAt, now),
            cancellationToken);
        _logger.LogInformation("Partition {Action} sent for {Table}", action, table.Table);
        return action == AlertAction.SendCritical ? "critical alert sent" : "recovery sent";
    }

    private static PartitionStatus Worst(IEnumerable<PartitionTableReport> tables)
    {
        var statuses = tables.Select(t => t.Status).Where(s => s !=
        PartitionStatus.Skipped).ToList();
        if (statuses.Count == 0)
        {
            return PartitionStatus.Skipped;
        }

        return statuses.Contains(PartitionStatus.Critical) ? PartitionStatus.Critical :
        PartitionStatus.Ok;
    }
}

```

ПРИЛОЖЕНИЕ В

Правила отправки оповещений

Класс `AlertPolicy` решает, отправлять ли `alert` или `recovery`, класс `AlertMessages` формирует текст сообщений (подраздел 2,2).

Листинг В.1 – `AlertPolicy.cs`

```
using System.Globalization;

namespace TaskManager.Application.Partitioning;

/// <summary>
/// What the partition check should send.
/// </summary>
public enum AlertAction
{
    /// <summary>
    /// Nothing: the state did not change or was already reported.
    /// </summary>
    None,

    /// <summary>
    /// Report missing partitions.
    /// </summary>
    SendCritical,

    /// <summary>
    /// Report that a previously reported problem is gone.
    /// </summary>
    SendRecovery
}

/// <summary>
/// Decides when to alert so the same problem is reported once, not on every check:
/// 08:00 CRITICAL → alert, 08:05 CRITICAL → nothing, 08:15 OK → recovery.
/// </summary>
public static class AlertPolicy
{
    /// <summary>
    /// Decides what to send for the current check result.
    /// </summary>
    /// <param name="previous">The state saved by the previous check, if any.</param>
    /// <param name="status">The status found now.</param>
    /// <param name="missingPartitions">The missing partitions found now, comma-
    separated.</param>
    public static AlertAction Decide(PartitionAlertState? previous, PartitionStatus status,
    string missingPartitions)
    {
        if (status == PartitionStatus.Critical)
        {
            // The same set of missing partitions that was already delivered is not reported
            again;
            // a new missing partition or an undelivered alert is.
            var alreadyReported = previous is { Status: PartitionStatus.Critical, NotifiedAt:
            not null }
            && previous.MissingPartitions == missingPartitions;
            return alreadyReported ? AlertAction.None : AlertAction.SendCritical;
        }

        // Recovery only makes sense after a CRITICAL alert that actually reached someone
        return status == PartitionStatus.Ok && previous is { Status: PartitionStatus.Critical,
        NotifiedAt: not null }
        ? AlertAction.SendRecovery
        : AlertAction.None;
    }
}
```

```

/// <summary>
/// Texts of partition alerts.
/// </summary>
public static class AlertMessages
{
    /// <summary>
    /// Builds the alert about missing partitions.
    /// </summary>
    public static string Critical(string table, IEnumerable<string> missing, string horizon,
DateTime checkedAt) =>
        $"🚨 Partition alert\n\nTable: {table}\n\nMissing partitions:\n{string.Join('\n',
missing)}\n\n" +
        $"Expected horizon: {horizon}\n\nChecked at:\n{Format(checkedAt)}";

    /// <summary>
    /// Builds the recovery message.
    /// </summary>
    public static string Recovery(string table, DateTime checkedAt) =>
        $"🟢 Partition check OK\n\nTable: {table}\n\nAll required partitions exist.\n\nChecked
at:\n{Format(checkedAt)}";

    private static string Format(DateTime moment) =>
        moment.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture) + " UTC";
}

```